From a product specification to the requirements it must fulfil to be compliant

01 Partner

Who we are. Certivity is a regulation-tech company in Munich. It collects regulations, standards and laws from official sources worldwide, converts them into structured text with paragraph-level citations and keeps every version.

Our end user. Engineers at companies that manufacture products and components and sell them in many countries.

What we want from this challenge. To see how far an agent can get on a task our users do by hand today, and to meet teams we would like to work with after the event.

02 Problem

The situation. Before a product goes to market, its engineers must know every requirement it has to fulfil under the regulations of each country where it will be sold. When a regulation changes, they must also find out which of those requirements change, and update the technical specification.

What breaks today. Regulations are long documents in many versions, spread over many sources. Engineers search them by hand and read hundreds of pages to find the few paragraphs that apply to their part. A search tool returns documents, not obligations. When a regulation is amended, nobody can tell which of the requirements already written are now out of date.

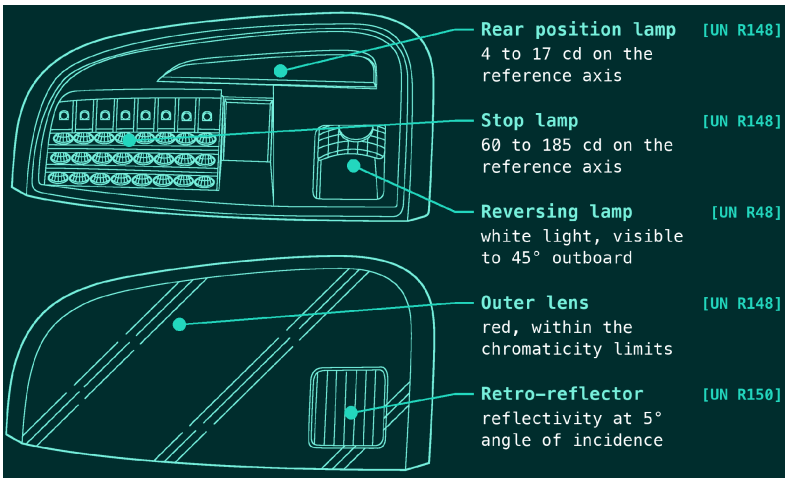
03 Definitions

- **UN Regulation (UN R):** a technical regulation for vehicles and their parts, published by UNECE, for example UN R16 (safety-belts and restraint systems).
- **Version:** each regulation exists in several versions (series and supplements).
- **Exploded view:** a drawing that shows the parts of an assembly pulled apart.
- **Requirement:** one engineering obligation the product must fulfil, such as a measurable limit.
- **Golden dataset:** the expected requirements for a case, prepared by Certivity's regulatory experts. You never see it for the held-out case.

04 Challenge

Build an agent or tool that takes a technical specification and/or an exploded view of a component or product, and produces the engineering requirements it must fulfil. You may use any tools, models and methods.

	Goal
Main goal (required)	**Input:** a specification and/or exploded view, for example a front bumper or a rear lamp. **Output:** a list of requirements. Each one cites the UN Regulation it comes from, its version and its page. A test case per requirement is optional.
Sub-goal (optional)	**Input:** the requirements derived from one version of a UN Regulation, and a later version of the same regulation. **Output:** which requirements must change (modified, removed or added), and what changed in the regulation to cause it.



Example input: an exploded view of a vehicle rear lamp, with illustrative requirements. Drawing: US design patent D592,336 S.

A good solution picks the regulations that apply to the product, writes one clear obligation per requirement in terms an engineer can act on, and cites a version and page that someone can open and check. Example (illustrative): for a rear lamp, "the rear position lamp shall give 4 to 17 cd on the reference axis", citing UN R148.

Recall matters somewhat more than precision. We measure both, but a missed requirement costs our users more than an extra one. An engineer can discard a requirement that does not apply while reviewing the list, but cannot act on one they never saw, and a missed requirement can mean a product that is not compliant. Do not pad the list to be safe, though: every extra requirement is something the engineer has to read and remove.

What you submit. Your submission has two parts. (1) **On the organizers' central platform:** answers to a few basic questions, a GitHub repository and a demo. (2) **On Certivity's scoring platform:** your output files (section 08).

05 Required

What a solution needs for the jury to consider it.

#	Element	Verified by
R1	A main-goal output file that passes `validate.py` and is uploaded for the held-out case `heldout-front-bumper`	Platform: file accepted and scored
R2	Every requirement cites a regulation from `regulations/index.json`, with its version label and a page number	Platform validation
R3	A demo that runs from input (specification or drawing) to requirements without manual steps	Demo in the pitch
R4	Answers to the basic questions, a GitHub repository and the demo, submitted on the organizers' platform	Organizers' platform

Worth exploring (hints, not required)

- Narrow down to the applicable regulations first, then read only those. The index lists more regulations than apply.
- Read the exploded view with a vision model to list parts and functions. Add a test case per requirement. Try the sub-goal: compare the two versions of a regulation and flag the requirements that change.

Not needed: building your own regulation database or crawler.

06 Data and resources

Item	Format, size	Link or access	Available from	NDA
Participant package: `README.md`, schemas, tools (`validate.py`, `score_local.py`, `fetch_regulations.py`), `regulations/index.json`	Public repository, small	`github.com/Certivity/industrial-ai-hackathon`	Before kickoff	None. Data for use in this hackathon only
UN Regulation PDFs	PDF, rendered by Certivity	Links in `index.json`; download with `python tools/fetch_regulations.py` (checksums verified)	At kickoff	None. Links expire after the event
Worked example: rear lamp, with its answer (`expected.json`)	Markdown, PNG, JSON	In the package: `cases/dev-rear-lamp/`	At kickoff	None
Worked example for the sub-goal: old and new version, old requirements, answer	PDF links, JSON	In the package: `subgoal/dev/`	At kickoff	None
Held-out case: front bumper (specification and exploded view), and the sub-goal pair	Second zip	Link announced on screen and by the organizers	Sunday 8:00	None

Public sources. UNECE vehicle regulations: https://unece.org/transport/vehicle-regulations (not downloadable by script, which is why we host the PDFs). Drawing of the worked example: US design patent D592,336 S, FIG. 5.

07 Evaluation

Finalists pitch on Sunday from 15:00 in three parts: the solution, a demo and proof. Certivity pre-selects about four finalist teams from the platform scores and the demos first. The selected teams then pitch their solution at the end and the jury picks one winner based on their combined scores, including the pitch.

Criterion	Weight	What the jury looks for
Score on the held-out case	40%	Recall and precision of your requirements against the golden dataset, at the level of regulations, pages and requirements. Ranking metric: F2 (recall weighted over precision), with recall as the tie-break

Criterion	Weight	What the jury looks for
Demo and user interface	20%	The flow runs end to end, an engineer can follow it, and every requirement links to its source page
Sub-goal result (optional)	15%	Recall of the requirements flagged for change, and the accuracy of the change stated for each one
Potential impact	15%	Would Certivity continue, test or deploy it
Pitch	10%	

Proof. The golden dataset for the held-out case is provided and kept by Certivity. You never receive it. The platform scores each upload against it and shows the headline numbers, never the matched pairs. The worked example comes with its answer so you can calibrate before Sunday. Held-out inputs are released on Sunday at 8:00.

A strong submission cites only regulations that apply, with the right version and page; writes one obligation per requirement; and shows in the pitch where it is weak and how that was measured.

A weak submission cites many regulations "to be safe" (precision drops), merges several obligations into one requirement (recall drops), cites the wrong version of a regulation, or has no working demo.

08 How to submit

1. Get the package from `github.com/Certivity/industrial-ai-hackathon` and run `python tools/fetch_regulations.py` to download the PDFs.
2. Build on the worked example. Check a file with `python tools/validate.py my_submission.json`, and compare regulation and page level offline with `tools/score_local.py`.
3. Register your team on the platform at https://hackathon.certivity.io/ with the invite code shown on the screen at the event: team name, 1 to 6 members (name and email) and a password of at least 10 characters. **Your team id is shown afterwards; put it in your files.**
4. Upload one JSON file per challenge (main goal or sub-goal) and case. Errors are shown with their JSON path. A valid file is scored in the background and its headline scores appear on your page. You can upload **5 valid files per hour for each challenge**; files that fail validation do not count.
5. **Sunday 8:00:** unzip the held-out package into the same folder, run `fetch_regulations.py` again, and upload for `heldout-front-bumper` (and `subgoal-heldout` if you do the sub-goal). **For the held-out case your last valid upload counts, not your best**, so do not upload a worse file at the end. Uploads close at 13.00.
6. Submit your GitHub repository and demo, and answer the basic questions, on the organizers' platform.